

Risk Management

AUTHOR
Annalisa Ori

Funzioni implementate per la risoluzione degli esercizi proposti nel corso di Risk Management

Implementiamo una funzione **PtoR** che trasforma una matrice dei prezzi in una matrice dei rendimenti netti, una funzione **mu** per il calcolo del vettore delle medie e una funzione **Sigma** per il calcolo della matrice di varianza-covarianza.

```
PtoR <- function(P,
                 n = 1)

# n = 1 rendimento semplice
# n = c rendimento continuamente capitalizzato

{
  P <- as.matrix(P)
  P2 <- P[2:nrow(P),]
  P1 <- P[1:(nrow(P) - 1),]
  if (n == "c") log(P2/P1) else ( (P2/P1)^(1/n) - 1 ) * n
}

#-----
mu <- function(X)

{
  if (is.list(X)) as.vector( X[[1]]) else colMeans(X)
}

#-----
Sigma<-function(X)

{
  if (is.list(X)) as.matrix( X[[2]]) else cov(X)
}
```

Sia α un portafoglio efficiente (non a varianza minima) e sia β un portafoglio generico. Ponendo

$$b_{\alpha} = \mu_{\alpha} - c\alpha\sigma_{\alpha}^2$$

posso scrivere

$$\mu_{\beta} = b_{\alpha} + c_{\alpha}\sigma_{\alpha,\beta}$$

valida per qualunque portafoglio β .

Poiché la relazione si può dimostrare essere necessaria e sufficiente, c'è una corrispondenza biunivoca tra il parametro c e il portafoglio efficiente α

Sviluppando i conti, posso scrivere la funzione "portfolio", che identifica il portafoglio efficiente in corrispondenza a un certo parametro c :

$$\alpha = \frac{\Sigma^{-1}m \mathbf{1}_N^{\top} \Sigma^{-1}m}{\mathbf{1}_N^{\top} \Sigma^{-1}m} \eta_c + \frac{\Sigma^{-1} \mathbf{1}_N}{\mathbf{1}_N^{\top} \Sigma^{-1} \mathbf{1}_N} (1 - \eta_c) = \alpha_M \eta_c + \alpha_{\min} (1 - \eta_c).$$

```

portfolio <- function(X,c)
{
  m <- mu(X)
  S <- Sigma(X)
  Sm <- solve(S,m) #S^-1 * m #affinchè le quote sommino a 1
  Se <- solve(S,rep(1,nrow(S))) #S^-1 * 1N
  eta <- sum(Sm) #1N' * S^-1 * m
  aM <- Sm/eta
  amin <- Se/sum(Se) #(S^1 * 1N)/(1N' * S^-1 * 1N)
  aM * eta/c + amin * (1-eta/c)
  #formula della frontiera efficiente
}

```

La funzione **frontier** costruisce la frontiera efficiente a partire da una matrice di rendimenti X . Dopo aver stimato il vettore delle medie μ e la matrice varianza-covarianza Σ , vengono selezionati due portafogli efficienti α_1 e α_2 .

Per il teorema di combinazione dei portafogli efficienti, ogni portafoglio efficiente può essere scritto come

$$r(t) = tr_{\alpha_1} + (1 - t)r_{\alpha_2}$$

$$\mu(t) = t\mu_{\alpha_1} + (1 - t)\mu_{\alpha_2}$$

$$\sigma^2(t) = t^2\sigma_{\alpha_1}^2 + (1 - t)^2\sigma_{\alpha_2}^2 + 2t(1 - t)\sigma_{\alpha_1, \alpha_2}$$

Variare il parametro t in un opportuno intervallo permette di tracciare l'intera frontiera efficiente.

Se `sd = TRUE`, allora sull'asse x verrà rappresentata la deviazione standard invece della varianza.

```

frontier <- function(X,
  x = NULL, #per fissare la lunghezza dell'asse x
  x0 = NULL,
  mesh = 100, #numero di portafolio usati per rappr
  sd = FALSE, #sull'asse x ho la varianza, non sd
  graph = TRUE,
  title = "Efficient frontier",
  output = FALSE)
{
  S <- Sigma(X)
  m <- mu(X)

  #due portafogli efficienti
  a1 <- portfolio(X,c = 1)
  a2 <- portfolio(X,c = 0.5)

  #le loro medie
  m1 <- as.numeric(a1%%m)
  m2 <- as.numeric(a2%%m)

  #var e cov
  s1 <- as.numeric(a1%%S%%a1)
  s2 <- as.numeric(a2%%S%%a2)
  s12<- as.numeric(a1%%S%%a2)
  ss <- diag(S)

  if (is.null(x)) x <- max(ss)

  x <- max(x,ss)

  if (is.null(x0)) x0 <- 0

```

```

#nuovo portafoglio come combinazione convessa dei due portafogli
#efficienti, sarà efficiente
A <- s1 + s2 - 2*s12
B <- s2 - s12
C <- s2 - 1.2*x
t1 <- as.numeric((B - sqrt(B^2 - A*C))/A)
t2 <- as.numeric((B + sqrt(B^2 - A*C))/A)
t <- t1 + (0:mesh)*(t2 - t1)/mesh
mt <- t*m1 + (1 - t)*m2
st <- t^2*s1 + (1 - t)^2*s2 + 2*t*(1 - t)*s12
xl <- "Var"

if (sd) {
  st <- sqrt(st)
  xl <- "Std. dev."
  x <- sqrt(x)
  x0 <- sqrt(x0)
  ss <- sqrt(ss)}

if (graph){
  plot(st,mt,
       type="l",main=title,
       xlab=xl,ylab="Mean",xlim=c(max(x0,min(st)),max(st)))
  points(ss,m)
}

if (output) cbind(st,mt)
}

```

La funzione seguente determina un portafoglio efficiente che soddisfa un vincolo assegnato, sulla media, sulla varianza, sul parametro c , sul termine b della retta di separazione, oppure sullo Sharpe ratio. A partire dalla stima della media μ e della matrice varianza-covarianza Σ , vengono selezionati due portafogli efficienti α_1 e α_2 , che generano l'intera frontiera efficiente.

Ogni portafoglio efficiente viene espresso come combinazione di questi e successivamente il parametro t viene determinato imponendo il vincolo richiesto (media, varianza, c , b o Sharpe ratio), ottenendo così il portafoglio efficiente corrispondente. La funzione restituisce i pesi del portafoglio e le principali quantità di interesse, e ne fornisce una rappresentazione grafica sulla frontiera efficiente.

In particolare coincide con portfolio per "what=c", per "what=mean" impone rendimento uguale al target fissato, per "what=var" impone varianza target, mentre con "what=sr" massimizza Sharpe Ratio. Infine con "what=b" massimizza

$$\mu - b * \sigma^2$$

```

portfolio.target<-function(X,
                           target = NULL,
                           what = c("mean","var","c","b","sr"),
                           r0 = 0,
                           graph = TRUE,
                           title = "Efficient frontier",
                           output = TRUE)
{
  S <- Sigma(X)
  m <- mu(X)

```

```

a1 <- portfolio(X,c=1)
a2 <- portfolio(X,c=0.5)

m1 <- as.numeric(a1%%m)
m2 <- as.numeric(a2%%m)

s1 <- as.numeric(a1%%S%%a1)
s2 <- as.numeric(a2%%S%%a2)
s12 <- as.numeric(a1%%S%%a2)

if (!is.null(target)) target <- as.numeric(target)

A <- s1 + s2 - 2*s12
B <- s2 - s12
C <- s2

tmin <- B/A
smin <- C - B^2/A

if (what=="mean"){
  t <- (target - m2)/(m1 - m2)
  a <- t*a1 + (1 - t)*a2
  c <- 1/(t + 2*(1 - t))
}

if (what=="var"){

  if (is.null(target)){
    a <- tmin*a1 + (1 - tmin)*a2
    c <- 1/(tmin + 2*(1 - tmin))}

  if (!is.null(target)){
    if (B^2<A*(C - target)) stop("Warning: target too low")
    t <- ( B - sqrt( B^2 - A*(C - target) ) )/A
    a <- t*a1 + (1 - t)*a2
    c <- 1/(t + 2*(1 - t))}
}

if (what=="b"){
  a <- solve(S,m-target)
  c <- sum(a)
  a <- a/c
}

if (what=="c") {
  a <- portfolio(X,c=target)
  c <- target
}

if (what=="sr"){

  if (is.null(target)){
    t <- ( (m1 - m2)*C + (m2 - r0)*B )/( A*(m2 - r0) + B*(m1 - m2) )
    a <- t*a1 + (1 - t)*a2
    c <- 1/(t + 2*(1 - t))
  }

  if (!is.null(target)){
    AA <- (m1 - m2)^2 - A*target^2

```

```

BB <- (m1 - m2)*(m2-r0) + B*target^2
CC <- (m2 - r0)^2 - C*target^2

if (BB^2<AA*CC) stop("Warning: target too high")

t1 <- -(BB + sqrt(BB^2 - AA*CC))/ AA
t2 <- -(BB - sqrt(BB^2 - AA*CC))/ AA
d1 <- ((t1*(m1 - m2) + m2 - r0) / sqrt(t1^2*A - 2*t1*B + C) - target)^2
d2 <- ((t2*(m1 - m2) + m2 - r0) / sqrt(t2^2*A - 2*t2*B + C) - target)^2

if (d1<d2) t <- t1

if (d2<=d1) t <- t2

a <- t*a1 + (1-t)*a2
c <- 1/(t + 2*(1-t))

if (c<0) stop("Warning: target too low")
}
}

ma <- as.numeric(a%%m)
sa <- as.numeric(a%%S%%a)

if (graph){
  frontier(X,x=sa,title=title,graph=graph)
  points(sa,ma,col="blue",pch=21,bg="red")
  if (c==Inf)
    abline(v = sa,col="magenta")
  else
    abline(a=ma-0.5*c*sa,b=c/2,col="magenta")
}

if (output) list(weights = a,
                 b = ma-c*sa,
                 c = c,
                 mean = ma,
                 var = sa,
                 sr = (ma - r0)/sqrt(sa))
}

```

La funzione “portfolio.opt” determina il portafoglio ottimale per un investitore con funzione di utilità $U(W, \mu, \sigma^2)$, a partire da una matrice di rendimenti X . Dopo aver stimato il vettore delle medie μ e la matrice varianza–covarianza Σ , il problema viene risolto massimizzando l'utilità lungo la frontiera efficiente.

Nel caso in cui non sia presente un asset risk–free, il portafoglio ottimale viene ottenuto massimizzando la funzione rispetto al parametro c che identifica i portafogli efficienti, individuando il punto di tangenza tra frontiera efficiente e curva di indifferenza.

Se è presente un asset privo di rischio con rendimento r_0 , viene dapprima determinato il portafoglio a massimo Sharpe ratio (portafoglio di mercato). Il portafoglio ottimale è quindi una combinazione del titolo risk–free e del portafoglio di mercato, ottenuta massimizzando l'utilità lungo la Capital Market Line.

```

portfolio.opt<-function(U,W,X,...,
                       title = "The optimal portfolio choice",
                       indiff = TRUE,
                       graph = TRUE,
                       r0 = NULL,
                       output = FALSE)

```

```

{
  m <- mu(X)
  S <- Sigma(X)

  if (is.null(r0)){
    F <- function(c,...){
      a <- portfolio(X,c)
      U(W,a%%m,a%%S%%a,...) }

    #ottimizzo la funzione

    H <- optimize(f = F,
                  interval = c(1e-05,1e+05),
                  ...,
                  maximum = TRUE)

    a <- portfolio.target(X,
                          target = H[[1]],
                          what = "c",
                          0,
                          graph,
                          title,
                          output = TRUE)

    if (graph&indiff){

      x0 <- 0.75*a$var
      x1 <- 1.25*a$var

      y0 <- uniroot(function(m)U(W,m,x0,...)-H[[2]],
                    interval = c(-1,1) + a$mean,
                    extendInt = "yes")[[1]]
      y1 <- uniroot(function(m)U(W,m,x1,...)-H[[2]],
                    interval = c(-1,1) + a$mean,
                    extendInt = "yes")[[1]]

      x <- seq(x0,x1,length.out = 1000)
      y <- seq(y0,y1,length.out = 1000)
      z <- outer(x,y,
                 FUN = function(x,y)U(W,y,x,...) - as.numeric(H[[2]]))
      contour(x,y,z,
              levels = 0,
              col = "green",
              add = TRUE,
              drawlabels = FALSE)
      #curva di indifferenza
    }

    if (output) Out <- list(weights = a$weights,
                            mean = a$mean,
                            var = a$var,
                            b = a$b,
                            c=H[[1]],
                            maxU=H[[2]])
  }

  if (!is.null(r0)){
    aSR <- portfolio.target(X,
                            what = "sr",
                            r0 = r0,
                            graph = FALSE,

```

```

        output = TRUE)
#t: % investita in asset risk-free
#1-t: investita nel portafoglio di mercato
F <- function(t) U(W,t*r0 + (1 - t)*aSR$mean,(1 - t)^2*aSR$var,...)
J <- optimize(f = F,
              interval = c(0,1),
              maximum = TRUE)

t <- J[[1]]
m_opt <- t*r0 + (1 - t)*aSR$mean
sd_opt <- abs(1 - t)*sqrt(aSR$var)
w <- c(t,(1 - t)*aSR$weights)
names(w) <- c("r0",names(aSR$weights))

if(graph) {
  frontier(X,
           sd = TRUE,
           title = title,
           x = max(sd_opt^2,aSR$var),
           x0 = 0)#sd_opt^2
  abline(a = r0,b = aSR$sr,col="blue")
  points(cbind(c(sqrt(aSR$var),sd_opt),c(aSR$mean,m_opt)),
         col = c("blue","blue"),
         pch = c(22,21),
         bg = c("red","green"))

  if (indiff){
    x0 <- 0.5*sd_opt
    x1 <- 2*sd_opt
    y0 <- uniroot(function(m)U(W,m,x0^2,...)-J[[2]],
                  interval = c(m_opt-10,m_opt))[[1]]
    y1 <- uniroot(function(m)U(W,m,x1^2,...)-J[[2]],
                  interval = c(m_opt,m_opt+10))[[1]]
    x <- seq(x0,x1,length.out = 1000)
    y <- seq(y0,y1,length.out = 1000)
    z <- outer(x,y,FUN = function(x,y)U(W,y,x^2,...))
    contour(x,y,z,
            levels = as.numeric(J[[2]]),
            col = "green",
            add = TRUE,
            drawlabels = FALSE)
    points(sd_opt,m_opt,
           col = "green",
           pch = 21,
           bg = "blue")
  }
  if (output) Out<-list(weights = w, #quote del portafoglio
                       mean = m_opt, #rendim atteso
                       sd = sd_opt,
                       SR_mean = aSR$mean,
                       SR_sd = sqrt(aSR$var), #media e dev standard del portafoglio di mercato
                       maxU = J[[2]]) #ut max ottenuta
}
}
if (output) Out
}

```

La funzione `co.var` risolve un problema di ottimizzazione media-varianza sotto vincoli lineari sui pesi del portafoglio. Data una matrice di rendimenti X , vengono stimati il vettore delle medie μ e la matrice varianza-covarianza Σ .

In assenza di un rendimento target, la funzione determina i portafogli a rendimento minimo e massimo compatibili con i vincoli assegnati. Se invece è fissato un rendimento target m_t , viene risolto il problema di Markowitz:

$\min_{\alpha} \alpha^T \Sigma \alpha$ soggetto a $\mathbf{1}^T \alpha = 1$, $\mu^T \alpha = m_t$,

con ulteriori vincoli lineari del tipo $M_b \alpha \geq \ell_b$.

La funzione restituisce la varianza minima raggiungibile e i pesi del portafoglio ottimale compatibili con i vincoli imposti.

Osserviamo che se $M_b = \text{NULL}$, è imposto come matrice identità.

```
co.var<-function(mt=NULL,
                X,
                lb=0,
                Mb=NULL)
{
  N <- ncol(X)
  m <- mu(X)
  S <- Sigma(X)

  if (is.null(Mb)) Mb <- diag(N)
  if (length(lb)==1) lb <- rep(lb,nrow(Mb))
  if (nrow(Mb)!=length(lb))stop("Warning: Mb and lb do not match")

  f.obj <- m
  f.con <- rbind(rep(1,N),Mb)
  f.dir <- c("=",rep(">=",nrow(Mb)))
  f.rhs <- c(1+2*N,lb+Mb%%rep(2,N))

  H0 <- lp("min",f.obj,f.con,f.dir,f.rhs)
  H1 <- lp("max",f.obj,f.con,f.dir,f.rhs)

  m0 <- H0$objval-2*sum(m)
  a0 <- H0$solution-rep(2,N)
  m1 <- H1$objval-2*sum(m)
  a1 <- H1$solution-rep(2,N)

  if (is.null(mt)) U <- list(a0=a0, m0=m0, s0=a0%%S%%a0,
                             a1=a1, m1=m1, s1=a1%%S%%a1)

  if (!is.null(mt)){

    if (mt>m1|mt<m0) U=list(var=Inf,weights=a0)

    else{
      H <- solve.QP(Dmat = S,
                    dvec = rep(0,N),
                    Amat = cbind( rep(1,N) , mu(X) , t(Mb) ),
                    bvec = c( 1 , mt , lb ),
                    #tutti alpha superiori a un certo lb

                    meq = 2)
      U=list(var=2*H$value,weights=H$solution)}
    }
  U
}
```

La funzione `co.frontier` costruisce la frontiera efficiente di portafogli sotto vincoli lineari sui pesi, utilizzando la funzione `co.var` per calcolare la varianza minima per ogni livello di rendimento target.

In pratica, si selezionano N asset con rendimenti X , si calcolano il portafoglio a rendimento minimo m_0 e massimo m_1 , e poi si suddivide l'intervallo $[m_0, m_1]$ in "mesh" punti T . Per ciascun T_i si risolve:

$\min_{\alpha} \alpha^{\top} \Sigma \alpha$ soggetto a $\mathbf{1}^{\top} \alpha = 1$, $\mu^{\top} \alpha = T_i$, $M_b \alpha \geq \ell_b$

ottenendo così le varianze V_i dei portafogli efficienti corrispondenti.

Se richiesto, la funzione traccia la frontiera efficiente e restituisce i dati dei portafogli estremi e delle varianze corrispondenti.

```
co.frontier<-function(X,
                      x=NULL,
                      x0=NULL,
                      lb=0,
                      Mb=NULL,
                      sd=FALSE,
                      mesh=100,
                      graph = TRUE,
                      output = FALSE)
{
  m <- mu(X)
  S <- Sigma(X)
  N <- ncol(X)

  U <- co.var(NULL,X,lb,Mb)

  T <- U$m0 + (U$m1 - U$m0)*(1:mesh)/(mesh+1)
  #ripeto per ogni valore di T
  V <- (1:mesh)

  for (n in seq(T)) V[n] <- co.var(mt=T[n],X,lb,Mb)$var

  V <- c(U$s0,V,U$s1)
  T <- c(U$m0,T,U$m1)

  if (sd) V <- sqrt(V)

  if (graph==TRUE){
    frontier(X,
             x,
             x0,
             sd=sd,
             title = "Efficient frontier with and without constraints")

    lines(V,T,col="red")

    points(cbind(V[c(1,102)],T[c(1,102)]),
           col="blue",pch=21,bg="green")
  }

  if (output==TRUE)list(data = cbind(V,T),a0 = U$a0,a1 = U$a1)
}
```

La funzione seguente calcola il quantile di ordine p dei rendimenti X utilizzando la teoria dei valori estremi (EVT, Extreme Value Theory).

Se non viene fornita una soglia u , questa viene fissata al 95° percentile delle perdite: cioè si considerano come “valori estremi” il 5% delle peggiori perdite.

Indicando con T il numero totale di osservazioni e con T_u il numero di perdite oltre la soglia u , si stima il parametro ξ di Hill come

$$\xi = \frac{1}{T_u} \sum_{i=1}^{T_u} \log \frac{-X_i}{u}, \quad X_i \leq -u$$

Il quantile di ordine p viene quindi calcolato tramite la formula:

$$q_p = -u \left(\frac{T_u/T}{p} \right)^\xi$$

dove p è il livello di probabilità desiderato. Se p è troppo piccolo rispetto alla proporzione di valori estremi T_u/T , viene corretto al massimo T_u/T per garantire coerenza.

```
#-----
q_EVT<-function(p,X,u = NULL)
  #quantile di ordine p, EVT
{
  if (is.null(u)) u <- as.numeric(quantile(-X,0.95,names=FALSE))
  #prende il 95° percentile delle perdite se u non è dato
  #oltre a questo valore: 5% delle perdite peggiori, oltre
  #questo valore considero le perdite come valori estremi
  T <- length(X)
  Tu <- length(X[X<=-u]) #rendimenti estremi
  xi <- mean(log(-X[X<=-u]/u)) #stima di Hill
  #+ è grande, + le perdite estreme sono probabili
  p[p>Tu/T] <- Tu/T
  -u*( (Tu/T)/p )^xi
}
#X:vettore dei rendimenti
#p: livello del quantile (0.01 per es)
#u:soglia
```

La funzione seguente calcola invece la **densità dei rendimenti estremi** utilizzando la teoria dei valori estremi (EVT).

Sia X il vettore dei rendimenti e u la soglia per identificare gli eventi estremi (default al 95° percentile delle perdite). Si calcolano:

$$T = \text{numero di osservazioni}, \quad T_u = \text{numero di perdite oltre la soglia } u, \quad \xi = \frac{1}{T_u} \sum_{i=1}^{T_u} \log \frac{-X_i}{u}, \quad X_i \leq -u$$

Per $t \leq -u$, la densità EVT è stimata come

$$f(t) = -\frac{1}{t\xi} \left(\frac{-t}{u} \right)^{-1/\xi}$$

altrimenti $f(t) = 0$ per valori meno estremi.

In sintesi, la funzione restituisce la **densità di probabilità per perdite estreme** al di sopra della soglia u , basata sulla stima della coda di Hill.

```
d_EVT<-function(t,X,u = NULL)
{
  if (is.null(u)) u <- as.numeric(quantile(-X,0.95,names=FALSE))
  T <- length(X)
  Tu <- length(X[X<=-u])
  xi <- mean(log(-X[X<=-u]/u))

  if (t<=-u) -(1/(t*xi))*(-t/u)^(-1/xi) else 0
}
```

La funzione seguente calcola la **funzione di distribuzione cumulativa (CDF)** dei rendimenti estremi utilizzando la teoria dei valori estremi (EVT).

Sia X il vettore dei rendimenti e u la soglia per identificare gli eventi estremi (default al 95° percentile delle perdite). Si calcolano:

T = numero di osservazioni, T_u = numero di perdite oltre la soglia u , $\xi = \frac{1}{T_u} \sum_{i=1}^{T_u} \log \frac{-X_i}{u}$, $X_i \leq -u$

Per $t \leq -u$, la CDF EVT è stimata come

$$F(t) = \left(\frac{-t}{u}\right)^{-1/\xi}$$

mentre per $t > -u$, $F(t) = 1$, cioè non si considerano eventi estremi inferiori alla soglia.

In sintesi, la funzione restituisce la **probabilità cumulativa di perdite estreme** oltre la soglia u , basata sulla stima della coda tramite il parametro di Hill ξ .

```
p_EVT<-function(t,X,u = NULL)
{
  if (is.null(u)) u <- as.numeric(quantile(-X,0.95,names=FALSE))
  T <- length(X)
  Tu <- length(X[X<=-u])
  xi <- mean(log(-X[X<=-u]/u))
  if (t<=-u) (-t/u)^(-1/xi) else 1
}
```

La funzione **q_CF** calcola il **quantile corretto per asimmetria e curtosi** di una distribuzione approssimativamente normale, utilizzando l'espansione di Cornish-Fisher.

Siano p il livello di quantile desiderato, z_1 la **skewness** e z_2 la **kurtosis**. Si parte dal quantile della normale standard

$$F = \Phi^{-1}(p)$$

e si applica la correzione di Cornish-Fisher:

$$q_p \approx F + \frac{z_1}{6}(F^2 - 1) + \frac{z_2}{24}(F^3 - 3F) - \frac{z_1^2}{36}(2F^3 - 5F)$$

in modo da tener conto di asimmetria (skewness) e coda (kurtosis) della distribuzione.

```
q_CF<-function(p,z1,z2)
{
  F <- qnorm(p)
  #assumo normalità

  F + (z1/6)*(F^2 - 1) + (z2/24)*(F^3 - 3*F) - (z1^2/36)*(2*F^3 - 5*F)
  #tengo conto di coda e asimmetria
}
```

La funzione **VaR** calcola il Value-at-Risk (VaR) di un portafoglio al livello di probabilità p , utilizzando tre approcci alternativi:

1. Simulazione a partire da una distribuzione Df :

Si generano campioni $X \sim Df$, si calcolano media m_X e deviazione standard s_X , e si normalizza il quantile corrispondente al livello p :

$$\text{VaR}_p = - \left(\frac{q_p(X) - m_X}{s_X} \sigma + m \right)$$

dove m e σ possono essere fissati.

2. Metodo basato su una funzione quantile Qf :

Si calcolano media μ e deviazione standard σ integrando Qf su $[0, 1]$:

$$\mu = \int_0^1 Qf(u) du, \quad \sigma = \sqrt{\int_0^1 Qf(u)^2 du - \mu^2}$$

e poi il VaR normalizzato:

$$\text{VaR}_p = -\frac{Qf(p) - \mu}{\sigma} \sigma + m$$

3. Metodo storico ponderato (EWMA) a partire da un vettore di rendimenti R :

Si calcolano pesi esponenziali con fattore di smorzamento λ , si ordina R e si seleziona il valore R corrispondente al percentile p :

$$\text{VaR}_p = -\min R_i, : F_i \geq p$$

dove F_i è la somma cumulativa dei pesi ordinati.

La funzione restituisce il VaR al livello di probabilità p come lista con elementi p e il valore calcolato. Il calcolo è vettorizzabile per più valori di p contemporaneamente.

```
VaR<-function(p,
              Df = NULL,
              Qf = NULL,
              R = NULL,
              lambda = NULL,
              m = NULL,
              sigma = NULL,
              ...)
{
  F<-function(p){
    if (!is.null(Df)){
      X <- Df(...)
      mX <- E(X)
      sX <- sd(X)
      if (is.null(m)) m <- mX
      if (is.null(sigma)) sigma <- sX
      V <- -( (q.l(X)(p) - mX)/sX * sigma + m )
    }

    if (!is.null(Qf)){
      mu <- integrate(f = function(p)Qf(p,...),
                     lower = 0,
                     upper = 1)[[1]]
      m2 <- integrate(f = function(p)Qf(p,...)^2,
                     lower = 0,
                     upper = 1)[[1]]
      sd <- sqrt(m2 - mu^2)

      if (is.null(m)) m <- mu
      if (is.null(sigma)) sigma <- sd

      V <- -( (Qf(p,...) - mu) / ( sd ) * sigma + m )
    }

    if (!is.null(R)){
      if(is.null(lambda)) lambda <- 1

      T <- length(R)
      w <- lambda^(T:1)
      w <- w/sum(w)
      e <- order(R)
      R <- R[e]
      w <- w[e]
```

```

    F <- cumsum(w)
    V <- -min(R[F>=p])
  }

  list(prob = p, VaR = V)
}
V<-Vectorize(F)
V(p)
}

```

La funzione CVaR calcola il **Conditional Value-at-Risk (CVaR)**, cioè il valore atteso delle perdite che eccedono il Value-at-Risk (VaR) al livello di probabilità p .

L'argomento x rappresenta la soglia di CVaR; se $x = \text{NULL}$, viene impostato pari al corrispondente VaR calcolato tramite la funzione VaR.

La funzione può operare secondo tre approcci:

1. Simulazione da distribuzione Df :

- Si standardizzano i campioni X con media m_X e deviazione standard s_X ,
- Si calcola il valore atteso condizionato delle perdite estreme:

$$\text{CVaR}_p = -\frac{\mathbb{E}[X \mathbf{1}_{\{X \leq -x\}}]}{\mathbb{P}(X \leq -x)}$$

2. Funzione quantile Qf :

- Si calcola la media e la deviazione standard integrando Qf
- Si definisce la funzione normalizzata $f(u)$ e si trova il punto u_p corrispondente a x ,
- Il CVaR è calcolato come valore atteso integrale delle perdite estreme:

$$\text{CVaR}_p = -\frac{\int_0^{u_p} f(u) du}{u_p}$$

3. Metodo storico ponderato (EWMA) da rendimenti R :

- Si calcolano pesi esponenziali w con fattore λ ,
- Il CVaR è la media ponderata delle perdite che superano la soglia $-x$:

$$\text{CVaR}_p = -\frac{\sum_{R_i \leq -x} w_i R_i}{\sum_{R_i \leq -x} w_i}$$

```

#-----
CVaR<-function(p,
               x = NULL,
               Df = NULL,
               Qf = NULL,
               R = NULL,
               lambda = NULL,
               m = NULL,
               sigma = NULL,
               ...)
#-----
# x è il CVaR threshold; se x= NULL è posto uguale al VaR corrispondente.
{
  CV<-function(x){

    if (!is.null(Df)){
      X <- Df(...)

```

```

mX <- E(X)
sX <- sd(X)
if (is.null(m)) m <- mX
if (is.null(sigma)) sigma <- sX
X <- ( (X - mX)/sX ) * sigma + m
ff<-function(y) y * (y<=-x)
V <- -E(X,ff)/p(X)(-x)
}

if (!is.null(Qf)){
  ffun<-function(u)Qf(u,...)
  mu <- integrate(f=ffun, lower=0,upper=1)[[1]]
  m2 <- integrate(function(p)ffun(p)^2, lower=0,upper=1)[[1]]
  sd <- sqrt(m2-mu^2)
  if (is.null(m)) m <- mu
  if (is.null(sigma)) sigma <- sd

  fun<-function(u) ( (ffun(u) - mu) / ( sd ) ) * sigma + m

  up <- uniroot(function(u)fun(u)+x,
                interval=c(1e-05,1-1e-05),
                extendInt="yes")[[1]]
  V <- -integrate(f=fun, lower=0, upper=up)[[1]]/up
}

if (!is.null(R)){

  if(is.null(lambda)) lambda <- 1

  T <- length(R)
  w <- lambda^(T:1)
  w <- w/sum(w)
  V <- -R[R<=-x]%*%w[R<=-x]/sum(w[R<=-x])
}

list(level=x,CVaR=V)
}
CVec<-Vectorize(CV)

if (is.null(x)) x<-as.numeric(VaR(p,Df, Qf, R, lambda,
                                m, sigma,...)[2,])
CVec(x)
}

```

La funzione EVaR calcola il **Expected Value-at-Risk** (EVaR) al livello di probabilità p , cioè il valore atteso condizionato delle perdite pesate in maniera asimmetrica tra guadagni e perdite.

Il calcolo può essere effettuato in tre modi:

1. Simulazione da distribuzione Df :

$$\text{EVaR}_p = -t^* \quad \text{dove } t^* \text{ annulla } h(t) = p, \mathbb{E}[(X - t)_+] - (1 - p), \mathbb{E}[(t - X)_+]$$

2. Funzione quantile Qf :

- Si normalizzano i quantili $Qf(u)$ con media μ e deviazione standard σ ,
- Si trova t^* che annulla la funzione asimmetrica $h(t)$ integrale sui quantili.

3. Metodo storico ponderato su rendimenti R :

- Si applicano pesi esponenziali w con fattore λ ,
- Si trova t^* che equilibra le perdite e i guadagni ponderati.

Il valore EVaR restituito è $-t^*$, vettorizzabile per più livelli p .

```
EVaR <- function(p,
                Df = NULL,
                Qf = NULL,
                R = NULL,
                lambda=NULL,
                m = NULL,
                sigma = NULL,
                ...)
{
  fpos <- function(y)y*(y>0)

  Hp<-function(t,p){
    if(!is.null(Df)){
      X <- Df(...)
      if (is.null(m)) m <- E(X)
      if (is.null(sigma)) sigma <- sd( X )

      X <- ( (X-E(X))/sd( X ) ) * sigma + m

      h <- p*E(X - t,fpos)-(1- p)*E(t - X,fpos)
    }

    if(!is.null(Qf)){
      FFUN<-function(u)Qf(u,...)

      mu <- integrate(f=FFUN, lower=0,upper=1)[[1]]
      m2 <- integrate(function(p)FFUN(p)^2,lower=0,upper=1)[[1]]
      sd <- sqrt(m2 - mu^2)

      if (is.null(m)) m <- mu
      if (is.null(sigma)) sigma <- sd

      FUN<-function(u) ( (FFUN(u) - mu) / ( sd ) ) * sigma + m

      fp<-function(u) fpos( FUN(u) - t )
      fn<-function(u) fpos( t - FUN(u) )

      hp <- integrate(fp,
                      lower=0,upper=1,
                      stop.on.error = FALSE)[[1]]
      hn <- integrate(FUN,
                      lower=0,upper=1,
                      stop.on.error=FALSE)[[1]]
      h <- p * hp - (1 - p) * ( hp + t - hn )
    }

    if (!is.null(R)){
      if (is.null(lambda)) lambda <- 1
      w <- lambda^(length(R):1)
      w <- w/sum(w)
      h <- p*(as.vector(fpos(R - t))%*%w) - (1 - p)*(as.vector(fpos(t - R))%*%w)
    }

    h
  }
}
```

```
Esc<-function(p){  
  e<-uniroot(function(t)Hp(t,p),  
             interval=c(-1,1),  
             extendInt="yes")[[1]]  
  list(p=p, EVaR=-e)  
}  
  
EV<-Vectorize(Esc)  
EV(p)  
}
```